

BattleOps Week3 Assignments

“Where Kubernetes security meets real production chaos.”

SECTION 1: ENVIRONMENT SETUP (MANDATORY BEFORE LABS)

1. Create the BattleOps Namespace

- `kubectl create ns battleops`
- `kubectl label ns battleops purpose=battle`

2. Install All Tools Used in Labs

Inside your terminal:

- `sudo yum install -y jq tree`
- `curl -sLO https://get.helm.sh/helm-v3.14.0-linux-amd64.tar.gz`
- `tar -xf helm-v3.14.0-linux-amd64.tar.gz`
- `sudo mv linux-amd64/helm /usr/local/bin/`

3. Clone the BattleOps Lab Repository

battleops/

```
|— psa/  
|— exploit/  
|— sa-escalation/  
|— secrets/  
|— cosign/  
|— misc/
```

THE DEVOPS WAR ROOM

Students must create this folder structure manually for muscle memory.

SECTION 2: LAB 1: PSA BREAK / FIX WAR ROOM

Objective: Understand how *real production workloads break* when PSA jumps from *baseline* → *restricted*.

STEP 1: Apply Restricted PSA

```
kubectl label ns battleops \
  pod-security.kubernetes.io/enforce=restricted \
  --overwrite
```

STEP 2: Deploy a Known-Bad Pod

```
kubectl apply -f - <<EOF
apiVersion: v1
kind: Pod
metadata:
  name: badpod
  namespace: battleops
spec:
  containers:
  - name: c
    image: busybox
    securityContext:
      privileged: true
      command: ["sleep","3600"]
EOF
```

INFRATHRONE

THE DEVOPS WAR ROOM

What should happen? Pod must fail admission.

Questions

- What error message do you see?
 - Which field violated PSA?
 - If this happened in production, how would you identify the pod causing issues?
 - How do you allow this pod without disabling PSA globally? (namespace exemptions)
-

Fix Task: Modify pod → remove privileged: `kubectl edit pod badpod -n battleops`

SECTION 3: LAB 2: Service Account Token Shows Real Cluster Weakness

Objective: Show how a compromised pod = compromised cluster.

STEP 1: Create an Overpowered SA

- `kubectl create serviceaccount attacker -n battleops`
- `kubectl create clusterrolebinding attacker-admin \ --clusterrole=cluster-admin \ --serviceaccount=battleops:attacker`

STEP 2: Mount SA Into Pod

```
kubectl run attacker-pod \ -n battleops \ --image=bitnami/kubectl \ --overrides='
{ "spec": { "serviceAccountName": "attacker" } }'
```

STEP 3: Inside Pod Try Cluster-Wide Access

- `kubectl get pods -A`
- `kubectl get secrets -A`
- `kubectl get nodes`

Questions

- What does this demonstrate in a real production attack?
 - Why must we never bind cluster-admin to an app SA?
 - What PSA enforcement would block privilege escalation?
-

Fix Task

Remove dangerous binding: `kubectl delete clusterrolebinding attacker-admin`

SECTION 4: LAB 3: Secrets War Room (The GitHub Leak Simulation)

Objective: *why base64 Kubernetes secrets are insecure and how a dev leak compromises everything.*

STEP 1: Create a Sensitive Secret

```
kubectl create secret generic db-pass \
  --from-literal=password='SuperSecret123' \
  -n battleops
```

STEP 2: How Easy It Is to Decode

```
kubectl get secret db-pass -n battleops -o jsonpath='{.data.password}' | base64 -d
```

Questions

- If this secret was pushed to GitHub by mistake, how would an attacker exploit it?
 - Why is encryption-at-rest not enough?
 - What happens if a service account can read secrets in another namespace?
-

Fix Task: Implement SealedSecrets

Apply controller (already installed in class cluster).

Create sealed secret: `kubectl create secret generic db-pass \`

```
  --from-literal=password='FixedByBattleOps' \ --dry-run=client -o yaml | \ kubeseal
  --format yaml > sealed.yaml
```

Apply: `kubectl apply -f sealed.yaml`

SECTION 5: LAB 4: Image Supply Chain Attack Simulation (Cosign)

Objective: Protect the cluster from pulling tampered images.

STEP 1: Sign an Image

- cosign generate-key-pair
- cosign sign docker.io/library/nginx:latest

STEP 2: Verify Signature

cosign verify docker.io/library/nginx:latest

STEP 3: Create a Policy That Blocks Unsigned Images

Kyverno example:

apiVersion: kyverno.io/v1

kind: ClusterPolicy

metadata:

name: block-unsigned-images

spec:

validationFailureAction: enforce

rules:

- name: check-signature

match:

resources:

kinds:

- Pod

verifyImages:

- image: "**"

key: "cosign.pub"

Questions

- Why does signing matter more than scanning?
 - What are the weaknesses of “latest tag”?
 - How does this stop a supply-chain compromise?
-

Fix Task

- Deploy unsigned pod → observe rejection
 - Sign → deploy again → observe success
-



SECTION 6: LAB 5: Node-Level Security Attack (Kernel Surface Reduction)

Objective: Reveal how seccomp & AppArmor prevent container escape.

STEP 1: Run Pod Without Seccomp

```
kubectrl run no-seccomp \
```

```
--image=busybox \
```

```
-- sh -c "sleep 3600"
```

STEP 2: Try Dangerous Syscall

Inside pod: strace uname

STEP 3: Enforce seccomp

securityContext:

seccompProfile:

type: RuntimeDefault

Redeploy → strace fails → security enforced.

Questions

- Why containers can still escape without LSMs?
- Why AppArmor + Seccomp = minimal kernel attack surface?

Fix Task: Patch pod to enforce RuntimeDefault.

SECTION 7: LAB 6: Identity Hardening & Zero-Trust

Objective: Teach how misconfigured identity = cluster takeover.

STEP 1: Create a Pod With Default SA

```
kubectl run default-sa \  
  --image=bitnami/kubectl \  
  -n battleops
```

STEP 2: Check Its Token

```
cat /var/run/secrets/kubernetes.io/serviceaccount/token
```

STEP 3: Attempt Privilege Escalation

- kubectl get secrets
- kubectl get pods -A
- Most will be denied.

Questions

- Why should every workload have its own SA?
 - Why disable automountServiceAccountToken?
 - What attack path does this block?
-

Fix Task

Patch deployment: automountServiceAccountToken: false



THE DEVOPS WAR ROOM